Συστήματα Μικροϋπολογιστών

2η Ομάδα Ασκήσεων

Νικόλαος Λοβέρδος (03120442)

Δημήτριος Μιχαλακόπουλος (03120229)

Άσκηση 1

(α) Στο αρχείο **Ex1.8085** του zip βρίσκεται ο κώδικας του προγράμματος και για τα τρία υποερωτήματα της άσκησης. Οι θέσεις μνήμης με αφετηρία την διεύθυνση 0900 H στις οποίες αποθηκεύονται διαδοχικά με αύξουσα σειρά οι αριθμοί 0-127 φαίνονται παρακάτω:

08EA	00	08EB	00	08EC	00	08ED	00	08EE	00	08EF	00	08F0	00	08F1	00	08F2	00	08F3	00	08F4	00	08F5	00	08F6	00
08F7	00	08F8	00	08F9	00	08FA	00	08FB	00	08FC	00	08FD	00	08FE	00	08FF	00	0900	00	0901	01	0902	02	0903	03
0904	04	0905	05	0906	06	0907	07	0908	08	0909	09	090A	0A	090B	0B	090C	0C	090D	0D	090E	0E	090F	0F	0910	10
0911	11	0912	12	0913	13	0914	14	0915	15	0916	16	0917	17	0918	18	0919	19	091A	1A	091B	1B	091C	1C	091D	1D
091E	1E	091F	1F	0920	20	0921	21	0922	22	0923	23	0924	24	0925	25	0926	26	0927	27	0928	28	0929	29	092A	2A
092B	2B	092C	2C	092D	2D	092E	2E	092F	2F	0930	30	0931	31	0932	32	0933	33	0934	34	0935	35	0936	36	0937	37
0938	38	0939	39	093A	3A	093B	3B	093C	3C	093D	3D	093E	3E	093F	3F	0940	40	0941	41	0942	42	0943	43	0944	44
0945	45	0946	46	0947	47	0948	48	0949	49	094A	4A	094B	4B	094C	4C	094D	4D	094E	4E	094F	4F	0950	50	0951	51
0952	52	0953	53	0954	54	0955	55	0956	56	0957	57	0958	58	0959	59	095A	5A	095B	5B	095C	5C	095D	5D	095E	5E
095F	5F	0960	60	0961	61	0962	62	0963	63	0964	64	0965	65	0966	66	0967	67	0968	68	0969	69	096A	6A	096B	6B
096C	6C	096D	6D	096E	6E	096F	6F	0970	70	0971	71	0972	72	0973	73	0974	74	0975	75	0976	76	0977	77	0978	78
0979	79	097A	7A	097B	7B	097C	7C	097D	7D	097E	7E	097F	7F	0980	00	0981	00	0982	00	0983	00	0984	00	0985	00
0986	00	0987	00	0988	00	0989	00	098A	00	098B	00	098C	00	098D	00	098E	00	098F	00	0990	00	0991	00	0992	00
0993	00	0994	00	0995	00	0996	00	0997	00	0998	00	0999	00	099A	00	099B	00	099C	00	099D	00	099E	00	099F	00

(β) Το πλήθος των δυαδικών ψηφίων '1' των παραπάνω δεδομένων δίνεται θεωρητικά από την σχέση: $0.5 * (7 * 128) = 448_{10} = 01C0_{16}$. Στο πρόγραμμα, αυτό γίνεται εκτελώντας δεξιές περιστροφές για καθένα από τα 7 ψηφία σε κάθε αριθμό και ελέγχοντας αν η τιμή του CY είναι ίση με '1'. Τυπώνοντας στα LED εξόδου τις τιμές των καταχωρητών B και C αντίστοιχα έχουμε:



Άρα η τιμή του καταχωρητή BC είναι: $\mathrm{BC} = (01)(11000000)_2 = (01)(C0)_{16} = 448_{10}$.

(γ) Το πλήθος των αριθμών μεταξύ του $10_{16}=16_{10}$ και του $60_{16}=96_{10}$ θεωρητικά είναι $81_{10}=51_{16}$. Στο πρόγραμμα, αυτό γίνεται αν για κάθε αριθμό κάνουμε διαδοχικά τις συγκρίσεις $10H \leq x_n$ και $x_n \leq 60H$. Τυπώνοντας στα LED εξόδου την τιμή του καταχωρητή D έχουμε:



Άρα τιμή του καταχωρητή D είναι: $\mathrm{D} = 01010001_2 = 81_{10}$.

Άσκηση 2

Στο αρχείο **Ex2.8085** του zip βρίσκεται το ζητούμενο πρόγραμμα μαζί με τα απαραίτητα σχόλια. Κάθε 0.1 sec γίνεται έλεγχος την θέσης του διακόπτη, επομένως κάθε 0.2 sec αν ενεργοποιηθεί το push-button, τα LED εξόδου ανάβουν για 20 δευτερόλεπτα και μετά σβήνουν. Αν ενδιάμεσα, όμως, ξαναενεργοποιηθεί το push-button, τότε ο χρόνος των 20 sec ανανεώνεται.

Άσκηση 3

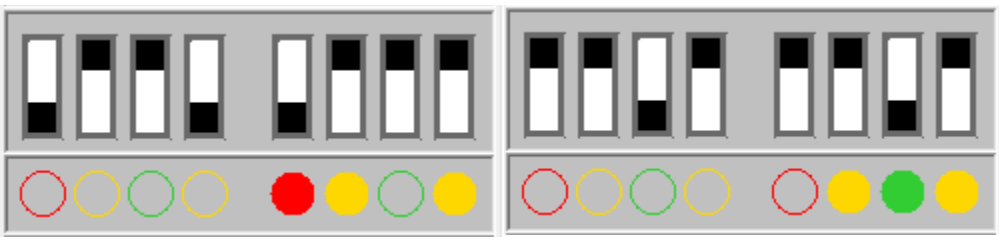
(i) Στο αρχείο **Ex3i.8085** του zip βρίσκεται το πρόγραμμα αυτού του υποερωτήματος μαζί με τα απαραίτητα σχόλια. Φορτώνουμε την είσοδο των dip switches και βρίσκουμε ποιο είναι το 1^{ο} δεξιότερο ON με διαδοχικές δεξιά περιστροφές. Έπειτα με διαδοχικές αριστερά περιστροφές ανάβουμε το αντίστοιχης τάξης LED. Το πρόγραμμα είναι συνεχούς λειτουργίας.

(ii) Στο αρχείο **Ex3ii.8085** του zip βρίσκεται το πρόγραμμα αυτού του υποερωτήματος μαζί με τα απαραίτητα σχόλια. Με την ρουτίνα KIND διαβάζουμε από το πληκτρολόγιο και ελέγχουμε ποιος αριθμός πατήθηκε από 1 έως 8. Έπειτα ανάβουμε το LED αντίστοιχης θέσης και όλα τα υψηλότερης τάξης LED μετά από αυτό. Το πρόγραμμα είναι συνεχούς λειτουργίας.

(iii) Στο αρχείο **Ex3iii.8085** του zip βρίσκεται το πρόγραμμα αυτού του υποερωτήματος μαζί με τα απαραίτητα σχόλια. Το πρόγραμμα βλέπει ποιο πλήκτρο πατάμε κάθε φορά, ελέγχοντας την κάθε γραμμή του πληκτρολογίου ξεχωριστά και προσπαθώντας να βρει σε ποια στήλη είναι το πατημένο πλήκτρο. Έπειτα φορτώνει τον κωδικό του πλήκτρου στην μνήμη και τον τυπώνει στο 7-segment display με τις ρουτίνες DCD και STDM. Το πρόγραμμα είναι συνεχούς λειτουργίας.

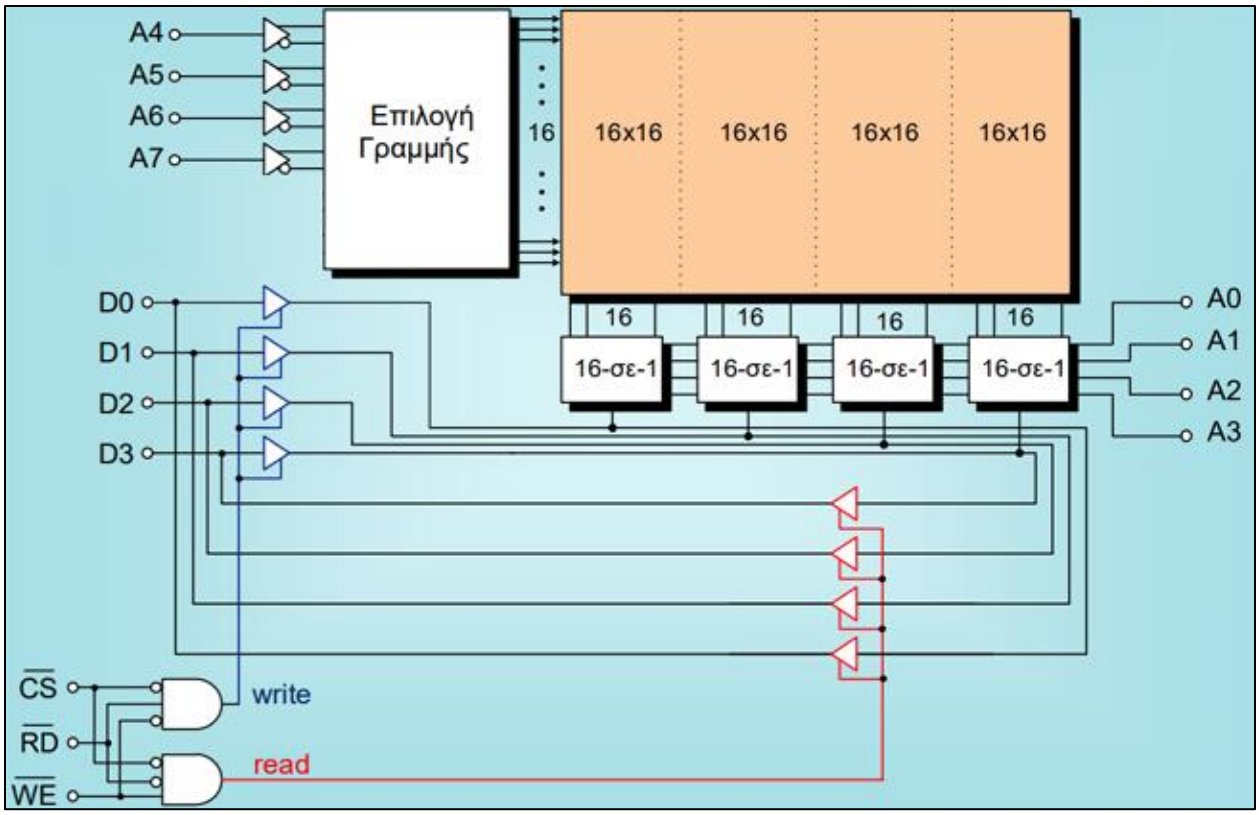
Άσκηση 4

Στο αρχείο **Ex4.8085** του zip βρίσκεται το ζητούμενο πρόγραμμα μαζί με τα απαραίτητα σχόλια. Φορτώνουμε την είσοδο των dip switches και με εντολές ANI θέλουμε να απομονώσουμε τα επιθυμητά ψηφία. Έπειτα με τις εντολές RRC και RLC επιχειρούμε να φέρουμε τα ψηφία στην επιθυμητή θέση και με τις εντολές ANA, ORA, XRA εκτελούμε τις ζητούμενες λογικές πράξεις. Η έξοδος του I.C. φορτώνεται στα 4 LSB LEDs με λογικές στάθμες ως εξής: ON => 1 και OFF => 0. Η ορθότητα του προγράμματός μας επιβεβαιώνεται από τα δύο παραδείγματα που φαίνονται παρακάτω:



Άσκηση 5

Η εσωτερική οργάνωση μια μνήμης SRAM 256x4 bit φαίνεται παρακάτω:



Η διαδικασία εγγραφής στην μνήμη ενεργοποιείται όταν έχουμε: $\overline{CS} = 0, \overline{WE} = 0$ *και* $\overline{RD} = 1$. Σε αυτή την περίπτωση η πάνω πύλη AND έχει έξοδο 1, η οποία ενεργοποιεί την επίτρεψη στα τέσσερα αριστερά buffer. Τα σήματα D0, D1, D2 και D3 περνάνε στους τέσσερις πολυπλέκτες 16-σε-1 (που εδώ λειτουργούν ως αποπλέκτες). Στην συνέχεια τα σήματα A0, A1, A2 και A3 που είναι οι επιλογείς των πολυπλεκτών καθορίζουν σε ποια εκ των 16 στηλών του κάθε μπλοκ θα γίνει η εγγραφή και τα σήματα A4, A5, A6 και A7 μέσω ενός αποκωδικοποιητή επιλέγουν μια από τις 16 γραμμές διεύθυνσης των μπλοκ. Έτσι, τα D0, D1, D2 και D3 γράφονται ταυτόχρονα το καθένα σε ένα από τα τέσσερα μπλοκ της μνήμης.

Η διαδικασία ανάγνωσης από την μνήμη ενεργοποιείται όταν: $\overline{CS} = 0, \overline{WE} = 1$ *και* $\overline{RD} = 0$. Σε αυτή την περίπτωση η κάτω πύλη AND έχει έξοδο 1, η οποία ενεργοποιεί την επίτρεψη στα τέσσερα δεξιά buffer. Τα σήματα D0, D1, D2 και D3 προκύπτουν από τις εξόδους των τεσσάρων πολυπλεκτών 16-σε-1. Ο κάθε πολυπλέκτης διαβάζει ένα bit του αντίστοιχου μπλοκ, η στήλη του οποίου καθορίζεται από τους επιλογείς A0, A1, A2 και A3 και η γραμμή από τα σήματα A4, A5, A6 και A7 μέσω ενός αποκωδικοποιητή. Έτσι, από κάθε μπλοκ διαβάζεται ταυτόχρονα ένα bit, τέσσερα στο σύνολο, τα οποία περνιούνται στα D0, D1, D2 και D3.

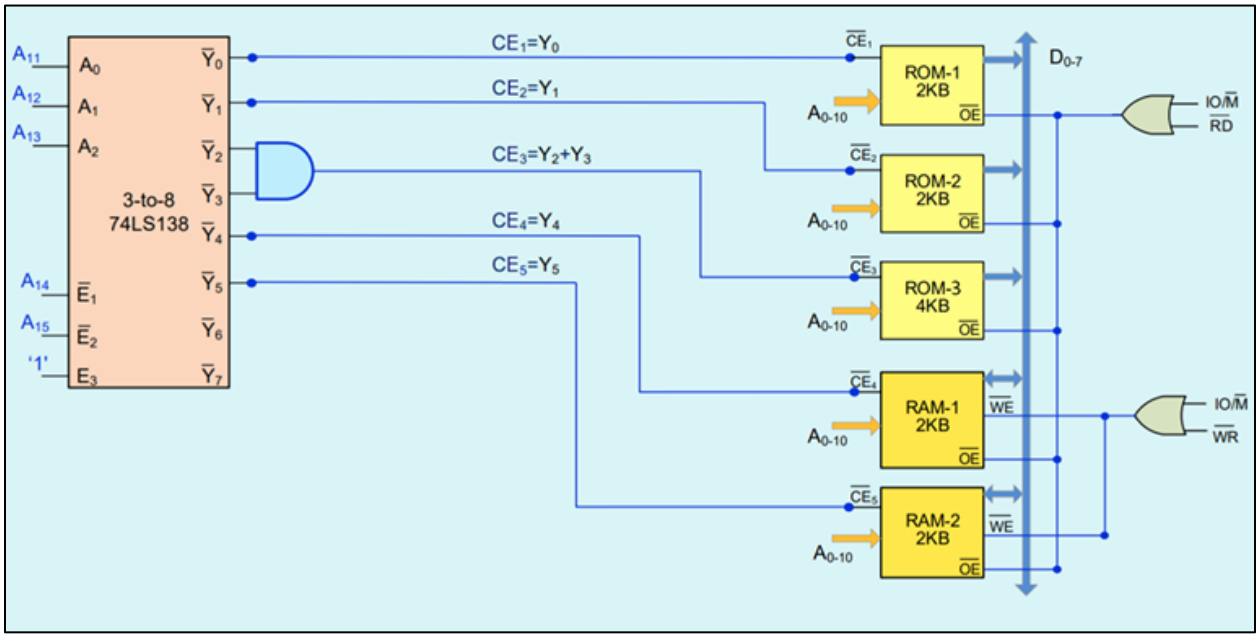
Για παράδειγμα, αν είχαμε την διεύθυνση (A7A6A5A4)(A3A2A1A0)=(0110)(1001), τότε στον πίνακα μνήμης επιλέγεται η 6^{η} γραμμή και η 9^{η} στήλη, είτε για ανάγνωση, είτε για εγγραφή.

Άσκηση 6

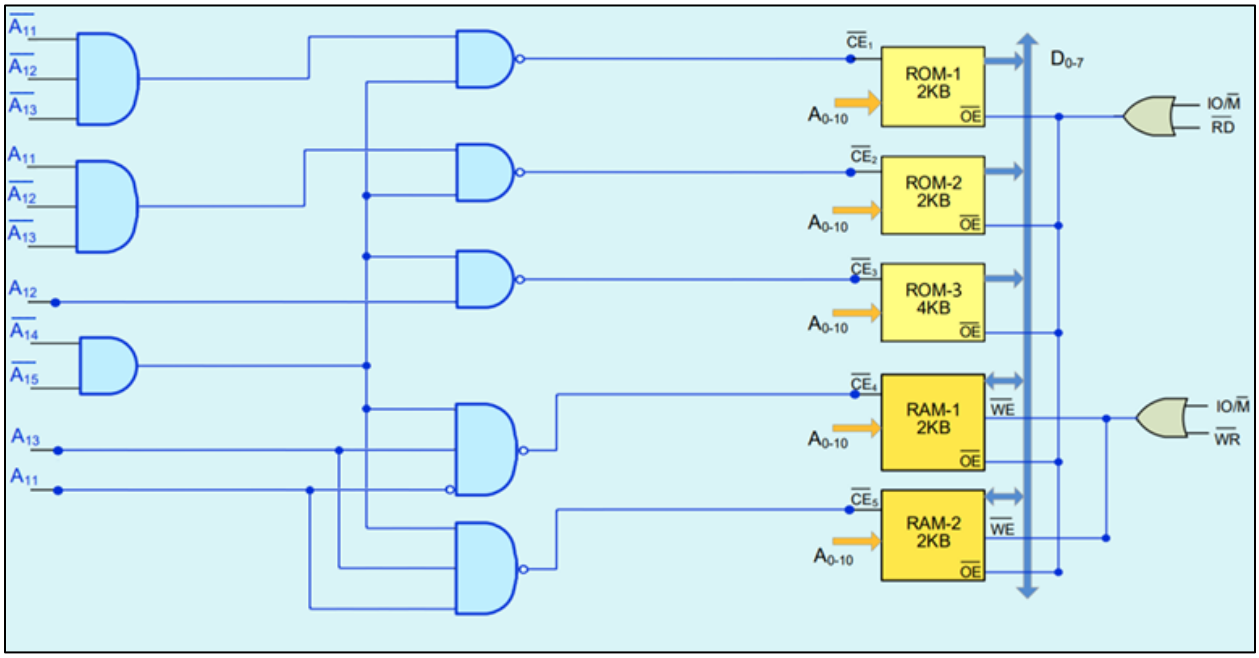
Ο χάρτης μνήμης του συστήματος μνήμης φαίνεται παρακάτω:

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00	Address	Memory
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000	ROM1-2K
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	07FF	
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0800	ROM2-2K
0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0FFF	
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1000	ROM3-4K
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1FFF	
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000	RAM1-2K
0	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	27FF	
0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	2800	RAM2-2K
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	2FFF	

(α) Το σύστημα μνήμης υλοποιημένο με αποκωδικοποιητή 3:8 και λογικές πύλες είναι το εξής:



(β) Το σύστημα μνήμης υλοποιημένο μόνο με λογικές πύλες είναι το εξής:



Άσκηση 7

Ο χάρτης μνήμης του μΥ-Σ 8085 φαίνεται παρακάτω:

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00	Address	Memory
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0000	ROM1-8K
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1FFF	
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2000	RAM1-4K
0	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	2FFF	
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	3000	RAM2-4K
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3FFF	
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4000	RAM3-4K
0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	4FFF	
0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	5000	R0M1-8K
0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	6FFF	

7000 Hex: θύρα εξόδου (Memory map I/O)

70 Hex: θύρα εισόδου (Standard I/O)

Το ζητούμενο μΥ-Σ 8085 που προκύπτει από τον παραπάνω χάρτη μνήμης είναι το εξής:

